Virtual memory & Paging

The problem: Limited RAM:-

Every program needs memory (RAM) to run..

Eg

Chrome, Terminal, vscode. → Each programer needs memory to store:
- Instruction
- variable
- stack
- heap.

But RAM is limited..

Eg

RAM = 8GB

But program might require → Chrome = 3GB, vscode = 2GB, Spam = 6GB ⇒ Total = 11GB

But RAM: only 8GB-

So how can os run program that need more memory than available RAM?

"This is where virtual memory comes in"

What is virtual memory ...?

virtual memory is a technique used by the operating system to use disk space as an extension of RAM.

"virtual memory is an memory management technique that allows program to use more memory than physically available by temporarily storing inactive memory pages on disk.

Simple → RAM + DISK = virtual memory

"DISK acts like extra memory"

"Swap space" ⇒ The disk area used as memory is called as swapspace.

Eg RAM = 8GB

DISK space = 6GB

Total system ⇒ 14GB.

Key idea of virtual memory:

The OS doesn't load the entire program into RAM.
Instead it loads only the parts currently needed.

Eg

Code
Function Stack
Data Heap
Libraries

But maybe only 10% of this is used at a time.
So the OS loads only that part into RAM.
The rest remain on disk.

Demand paging

virtual memory uses a technique called as Demand paging.

A page of memory is loaded only when the program need it.

Eg

Program need function → calculate tax()
&
OS loads only the memory containing the function
other functions stay on disk....

Virtual address space

Each program think it has its own large memory space

Program may think it has → 4GB Memory

But actual RAM may be

8GB Shared between many processes

This illusion is created virtual address space..

But actual memory is managed by the OS

virtual memory	(vs)	physical memory
Address used by program.		Actual RAM hardware.
virtual address → 0X2345		physical address → RAM location

Key Points

- → By virtual memory program bigger than RAM can run
- → Multiple program can run simultaneously
- → process Isolation
- → Efficient memory usage. [active memory stays in RAM].

Page fault ⇒ If the cpu request memory that is not in RAM then page fault occurs..

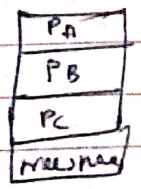
OS must:

- Pause process
- Load page from Disk
- Update memory mapping
- resume process

Paging :-

Before paging, [OS] used contiguous memory allocation, where a process had to be stored in one continuous block of RAM

RAM



Sometimes there is enough total free memory but not enough continuous memory

- This can fragmentation

Paging solve this problem

Paging ⇒ Paging divides memory into small fixed-size block.

Pages → Piece of a program in virtual memory
Frames → Piece of physical RAM

Program will be divided into pages

Suppose program 16KB

Page size = 4KB.

Program memory

Page 0
Page 1
Page 2
page 3

RAM divided into frames

RAM is divided into equal block

RAM
Frame 0
Frame 1
Frame 2
Frame 3
Frame 4

Now

Page to frame mapping

Page 0 → Frame 3
Page 1 → Frame 0
Page 2 → Frame 5
Page 3 → Frame 2

Page Table

To keep Track of when page are stored where

Page Table

Page	Frame
0	3
1	0
2	5
3	2

Page fault:

Sometimes the requested page is not in RAM
It might be stored on disk
when this → Page fault occurs

Steps

- CPU detects Page fault
- OS pauses the process
- required page is loaded from disk
- page table updated
- process resumes execution.

Scanned with OKEN Scanner